Mayo Clinic

Digital Product Engineering — Test 4

AI / ML Engineering Assessment — Project Narrative

Candidate: Hameem Mahdi M.S.E., Ph.D.

Submission · March 5, 2026

1. Executive Summary

This document provides a comprehensive narrative for the five-task AI/ML engineering assessment submitted as part of the Digital Product Engineering (DPE) hiring process. The solution is implemented in a single, self-contained Python script (solution.py) that generates all required submission artefacts in a single deterministic run.

The assessment covers the full breadth of production-grade ML engineering: model training with reproducibility controls, responsible-AI fairness evaluation, data-drift detection for production monitoring, retrieval-augmented generation (RAG), and latency profiling. Every artefact is generated from scratch by running python solution.py from the engineering_test_4/ directory.

Task	Topic	Key Result
1	Deterministic Structured ML	ROC-AUC 0.6231 · F1 0.3467 · Brier 0.2336
2	Fairness Evaluation	Dem. parity diff 0.218 · Equal opp. diff 0.233
3	Drift Detection	Max PSI 0.037 (stable) · KL-label 0.025
4	Mini-RAG Retrieval	Recall@3 0.267 · Exact Match 0.000
5	Latency Optimisation	avg ~0.019 ms/query · p95 ~0.034 ms/query

2. Project Structure and Environment

The repository is organised around a single engineering exercise directory. All source code lives in solution.py; all generated outputs are written to the submission/ subdirectory. No external model servers, databases, or cloud calls are required — the solution runs entirely offline with three standard libraries.

2.1 Directory Layout

```
engineering_test_4/
  solution.py              ← single entry-point, generates all artefacts
  submission/
    predictions.csv
    metrics.json
    fairness.json
    drift.json
    rag_results.json
    latency.json
    model_weights.bin
    RUN.md
  Archive/
    datasets/              ← provided (read-only)
    baseline_runner.py     ← reference only, not part of submission
```

2.2 Dependencies

The solution deliberately restricts itself to three mature, widely-available Python libraries so that any evaluator can reproduce the run with a single pip install command:

Library	Version (tested)	Purpose
numpy	≥ 1.24	Array maths, random seed, embedding dot-products
pandas	≥ 2.0	CSV I/O, data wrangling, group operations
scikit-learn	≥ 1.3	Pipeline, imputation, scaling, logistic regression, metrics

3. Task 1 — Deterministic Structured ML

Task 1 trains a binary classifier to predict 30-day hospital readmission from structured clinical features. The primary engineering goals are correctness, reproducibility, and separation of training and test-set transformations (no leakage).

3.1 Dataset

The dataset (patient_readmission.csv) contains 10,000 patient records with 10 clinical features: age, bmi, systolic_bp, diastolic_bp, hba1c, prior_admissions, smoker, sex, sdoh_index, and notes_length. The target column is readmitted_30d (binary: 0 = no readmission, 1 = readmission within 30 days).

The dataset has a significant class imbalance: only ~18.5% of patients are re-admitted within 30 days. This imbalance is one of the most important design considerations addressed in this task.

3.2 Train / Test Split

An 80/20 stratified split is applied using random_state=42. Stratification ensures the positive-class rate is preserved in both partitions, producing 8,000 training records and 2,000 test records.

3.3 Pre-processing Pipeline

A scikit-learn Pipeline wrapping a ColumnTransformer is used to guarantee that all imputation and scaling statistics are learnt exclusively from training data and then applied to the test set — the canonical approach to preventing data leakage:

Step 1 — Median Imputation: Missing values (present in hba1c and a few other columns) are replaced with the column median, computed on training data only.

Step 2 — Standard Scaling: Each numeric feature is z-score normalised (mean 0, std 1) using statistics from the training set. This prevents the model from being influenced by scale differences between features.

3.4 Model: Logistic Regression

Logistic Regression is selected as a transparent, well-understood baseline that is well-suited to a structured tabular dataset of this size. Its probabilistic output (y_score) is directly usable for fairness analysis and drift monitoring in subsequent tasks.

The full parameter configuration is:

Parameter	Value	Rationale
solver	lbfgs	Efficient L-BFGS optimiser; default for small datasets
max_iter	2000	Ensures full convergence; avoids ConvergenceWarning
random_state	42	Ties solver RNG to global SEED for reproducibility
class_weight	balanced	See §3.5 — addresses critical class imbalance
C (penalty)	1.0	Default regularisation; no hyperparameter search needed

3.5 Critical Design Decision: class_weight='balanced'

This is the most consequential design decision in the entire solution. Without class weighting, the model was empirically verified to be degenerate: the maximum predicted probability of any test-set patient plateaued at 0.4986 — just below the 0.5 classification threshold. This caused all 2,000 hard predictions to be $\hat{y} = 0$, yielding F1 = 0.000. The model appeared well-calibrated by ROC-AUC (≈ 0.62) but was operationally useless.

Root cause: the ~18.5% positive-class frequency causes the cross-entropy loss to be dominated by the majority class. The L-BFGS solver finds a minimum by skewing the decision boundary so aggressively toward the negative class that no patient exceeds the 0.5 probability threshold.

Fix: class_weight='balanced' instructs scikit-learn to weight each sample by the inverse class frequency, making the effective class balance 50/50 during optimisation. This is the standard, spec-compliant approach to class imbalance in logistic regression — it changes no other required parameter and the run remains fully deterministic with SEED=42.

Metric	Without class_weight	With class_weight='balanced'
F1 Score	0.0000 (degenerate)	0.3467 (meaningful)
ROC-AUC	0.6231	0.6231 (unchanged)

3.6 Outputs

Task 1 produces three artefacts:

File	Content
predictions.csv	2,000 rows: patient_id, y_true, y_score, y_pred
metrics.json	roc_auc, f1, brier, confusion_matrix, n_train, n_test, model_sha256
model_weights.bin	Deterministic pickle of the full sklearn Pipeline

3.7 Performance Results

Metric	Value
ROC-AUC	0.6231
F1 Score	0.3467
Brier Score	0.2336
Confusion Matrix	TN=989 FP=642 FN=157 TP=212
Training samples	8,000 (80%) | Test samples: 2,000 (20%)

4. Task 2 — Fairness Evaluation

Task 2 evaluates group fairness of the trained classifier with respect to the smoker attribute (binary: 0 = non-smoker, 1 = smoker). This reflects a Responsible AI concern: if the model systematically disadvantages or advantages a patient subgroup, it may amplify health inequities.

4.1 Fairness Metrics

Demographic Parity Difference

Measures the absolute difference in the rate at which each group receives a positive prediction ($\hat{y} = 1$), regardless of the true label. A value of 0 means both groups are predicted positive at equal rates.

```
DP_diff = |P(ŷ=1 | smoker=1) − P(ŷ=1 | smoker=0)|
```

Equal Opportunity Difference

Measures the absolute difference in True Positive Rate (sensitivity / recall) between groups. Focuses on whether the model is equally good at identifying truly at-risk patients across subgroups.

```
EO_diff = |TPR(smoker=1) − TPR(smoker=0)|
```

4.2 Results and Interpretation

Metric	Value	Interpretation
Dem. parity diff	0.2176	Smokers predicted positive 21.8 pp more often
Equal opportunity diff	0.2328	Model detects at-risk smokers 23.3 pp more often
Positive rate (non-smoker)	0.3814	38.1% of non-smokers predicted readmitted
Positive rate (smoker)	0.5990	59.9% of smokers predicted readmitted
TPR (non-smoker)	0.5146	51.5% of truly positive non-smokers detected
TPR (smoker)	0.7474	74.7% of truly positive smokers detected

Both fairness gaps exceed 0.20 — a commonly cited threshold for clinically meaningful disparity. The model is substantially more sensitive to readmission risk in smokers than in non-smokers. This likely reflects a genuine association in the training data (smokers are clinically higher-risk) rather than a modelling artefact, but it warrants monitoring in production to ensure the disparity does not widen or cause unintended clinical consequences.

4.3 Implementation Notes

The smoker attribute is joined to the predictions DataFrame via patient_id merge (not index alignment), ensuring exact row correspondence even after sorting. This is a subtle but important correctness detail when merging two independently-sorted DataFrames.

5. Task 3 — Drift Detection

Task 3 quantifies how much the feature distributions and label distribution have shifted between the baseline (training) cohort and a production-shifted cohort. This mirrors a critical production MLOps

concern: models trained on historical data may degrade when deployed to patients with different characteristics.

5.1 Population Stability Index (PSI)

PSI measures covariate drift for continuous features by comparing histogram bucket proportions between two distributions. It is computed for four features: age, bmi, systolic_bp, and hba1c.

```
PSI = Σ (q_i − p_i) × ln(q_i / p_i)
```

where p_i are baseline proportions and q_i are shifted proportions in each of 10 quantile bins derived from the baseline distribution. An epsilon of 1e-6 prevents log(0) instability. Standard interpretation thresholds are:

PSI Range	Interpretation
< 0.10	No significant shift — model likely stable
0.10 – 0.20	Moderate shift — monitor closely
> 0.20	Significant shift — consider retraining

5.2 KL Divergence (Label Shift)

Label shift — a change in the prior probability of the positive class — is measured using KL divergence between two Bernoulli distributions (one per dataset). The shifted dataset has a higher readmission rate, meaning the model may be systematically under-predicting risk in the production cohort.

```
KL(shifted || baseline) = p·ln(p/q) + (1−p)·ln((1−p)/(1−q))
```

5.3 Results

Metric	Value	Status
PSI — age	0.0247	Stable (< 0.10)
PSI — bmi	0.0191	Stable (< 0.10)
PSI — systolic_bp	0.0365	Stable (< 0.10)
PSI — hba1c	0.0369	Stable (< 0.10)
KL divergence (label)	0.0254	Mild label shift
Baseline positive rate	0.1847	18.47% of baseline patients readmitted
Shifted positive rate	0.2770	27.70% of shifted patients readmitted

All four PSI values fall comfortably below 0.10, indicating that the covariate distributions are stable. However, the label shift is notable: the readmission rate increases from 18.5% to 27.7% (+9.2 pp) in the shifted cohort. This is a meaningful signal that the production population is clinically higher-risk, and the model's calibration should be re-evaluated against the shifted cohort before deployment.

6. Task 4 — Mini-RAG Retrieval

Task 4 implements a Retrieval-Augmented Generation (RAG) retrieval layer using pre-computed dense embeddings. Given a set of clinical queries, the system retrieves the three most semantically similar documents from a corpus of 300 documents and evaluates retrieval quality against ground-truth labels.

6.1 Corpus and Embeddings

Asset	Shape	Description
doc_embeddings.npy	(300, 384)	L2-normalised float32 document embeddings
query_embeddings.npy	(30, 384)	L2-normalised float32 query embeddings
rag_qa.csv	30 rows	Ground-truth top-3 doc IDs per query

6.2 Retrieval Algorithm

Because both document and query embeddings are already L2-normalised (each vector has unit Euclidean length), cosine similarity reduces to a simple dot product:

```
similarity_matrix = query_embeddings @ doc_embeddings.T   # shape: (30, 300)
```

The top-3 document indices per query are identified with argsort and converted to 1-indexed IDs to match the ground-truth format. This brute-force batched matrix multiply is optimal at this scale (300 × 30 = 9,000 scores) — any approximate nearest neighbour (ANN) index such as FAISS HNSW would introduce overhead without any recall benefit for a 300-document corpus.

6.3 Evaluation Metrics

Recall@3: The fraction of ground-truth relevant documents that appear in the top-3 retrieved set. Averaged across all 30 queries.

Exact Match: The fraction of queries where the retrieved top-3 set exactly equals the ground-truth top-3 set (unordered).

6.4 Results

Metric	Value
Recall@3	0.2667 (8 of 30 queries retrieved ground-truth top-1 in their top-3)
Exact Match	0.0000 (no query's top-3 exactly matched the ground-truth top-3)

The moderate Recall@3 and zero Exact Match indicate that the embedding model captures partial semantic overlap but does not reproduce the ground-truth ranking precisely. This is expected behaviour for a general-purpose sentence embedding model applied to a specialised clinical document corpus — domain-tuned embeddings or a re-ranking layer would improve recall in a production system.

7. Task 5 — Latency Optimisation

Task 5 benchmarks the per-query retrieval latency of the RAG pipeline and reports average and 95th-percentile latency. The goal is to demonstrate awareness of production performance requirements and to contextualise the current implementation's efficiency.

7.1 Benchmarking Methodology

Key design choices in the latency measurement:

- **Timer precision:** time.perf_counter() is used (nanosecond resolution on modern hardware), not time.time() which has lower resolution on Windows.
- **Iteration count:** 100 full-batch iterations are timed. This provides a robust estimate and stable p95 computation.
- **Per-query latency:** Each iteration times the retrieval of all 30 queries as a single batched matrix multiply. The total batch time is divided by 30 to derive per-query latency.
- **Warmup:** No explicit warmup pass is required since the batched matrix multiply does not involve JIT compilation or lazy loading in NumPy.
- **Metric: p95:** The 95th-percentile is reported alongside the mean to capture tail latency, which is more operationally relevant than the average alone.

7.2 Results

Metric	Typical Value
avg_ms_per_query	~0.013 – 0.044 ms (varies with CPU load)
p95_ms_per_query	~0.017 – 0.073 ms (varies with CPU load)

Note: latency.json values vary across runs because wall-clock timing is inherently affected by OS scheduling and CPU thermal state. Only the structural invariants (avg > 0, p95 ≥ avg) are guaranteed to hold on every run — the absolute values are expected to fluctuate.

7.3 Scaling Analysis and Recommendations

The current brute-force approach (O(N) per query, where N = corpus size) is optimal for N = 300. For larger corpora the following migration path is recommended:

- **N ≈ 10,000:** Brute-force on CPU still sub-millisecond; no index needed.
- **N ≈ 100,000:** FAISS IVF-Flat (inverted file index) reduces search to O(N/n_cells). Recall ≥ 0.99 is achievable.
- **N ≈ 1,000,000+:** FAISS HNSW (hierarchical navigable small world graph) yields O(log N) query time. Minimal recall degradation (< 1%) at scale.
- **GPU deployment:** FAISS on GPU (or cuVS) provides 10–100x throughput improvement for very large corpora or high-QPS requirements.

8. Reproducibility

Reproducibility is a first-class requirement of the assessment. The solution implements five independent controls that together guarantee bit-identical output across any number of sequential runs in the same Python environment.

Control	Setting	Guarantees
Global RNG seed	np.random.seed(42)	NumPy random operations produce identical sequences
Train/test split RNG	random_state=42	Same 8,000/2,000 partition on every run
Model RNG	LogisticRegression(random_state=42)	Solver initialisation is deterministic
Pickle protocol	pickle.HIGHEST_PROTOCOL	Serialised model bytes are reproducible

SHA-256 self-check	model_sha256 in metrics.json	Evaluator can verify model integrity instantly

Verification: Two independent end-to-end runs were executed and compared. The SHA-256 hashes of model_weights.bin and predictions.csv were identical across both runs:

```
  model_weights.bin SHA-256 :
1cb64da18c5b5b84d410d26e1d036220063fecd61b8e5f68848dd263a79f21a8
  predictions.csv  SHA-256  :
ba8714c30b4135b5116c41b5c4ef68b90f971191472976cc4136493d35291a03
```

All 25 comparable numeric values in metrics.json, fairness.json, drift.json, and rag_results.json also matched exactly.

9. Submission Artefacts Summary

File	Task	Size	Description
predictions.csv	1	58,179 B	2,000 test-set rows: patient_id, y_true, y_score, y_pred
metrics.json	1	330 B	ROC-AUC, F1, Brier, confusion matrix, split sizes, SHA-256
model_weights.bin	1	2,646 B	Deterministic sklearn Pipeline (pickle)
fairness.json	2	280 B	Dem. parity diff, equal opp. diff, group rates
drift.json	3	263 B	PSI (4 features), KL-label, positive rates
rag_results.json	4	1,723 B	Top-3 doc IDs for 30 queries, Recall@3, Exact Match
latency.json	5	414 B	avg_ms_per_query, p95_ms_per_query, methodology notes
RUN.md	—	4,424 B	Reproduction instructions and design notes

10. How to Reproduce

The complete submission can be regenerated from scratch with the following four commands:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -U pip numpy pandas scikit-learn
python solution.py
```

All eight submission files will be written to the submission/ directory. The run completes in under five seconds on a modern laptop. No internet access, GPU, or additional configuration is required.